Colin Fergusson
3664465
CS4745
Assignment #1

A)

1. I parallelized every loop inside of the mode functions I made, the only loops I didn't parallelize were the ones inside main which I didn't parallelize because they weren't being timed. I parallelized all the loops inside the mode functions because they would run faster if every loop was parallelized rather than just the required ones.

2. For the matrix multiplication I always set the input arrays (A, and B) as well as the output array (C) as shared, then I set the loop indicies (i,j,k) as private. For the sum, max, and checksum I had C shared since it was the only array needed, and then the loop indicies (i,j) were private. I had the arrays shared since each individual thread needed to access each array to either read or write to it, then the loop indicies were private because having them shared would cause the threads to interfere with each other.

3. For sum and max I almost exclusively used reduction to calculate them because it is faster to do it that way for tasks like summing an array, or getting the maximum value as well as it being very easy to implement. The only times I didn't use reduction was when the mode that was being used handled the calculations itself (ex. modes 3 and 4).

4. I didn't use atomic or critical anywhere aside from in 3A and 3B where we were told to. The reason I didn't use them elsewhere was because I used reduction to calculate things instead.

5. I chose a static schedule for the matrix multiplication because the calculations weren't dependent on any other threads and it was the same calculation every time. For the sum, max, and checksum I used a dynamic schedule because they were all in the same loop so the work wouldn't always be the same and the results were dependent on what other threads did. I chose a chunk size of 16 mainly because I was primarily testing using N = 256 and 16 is a good size for that. Because mode 1 said to use two different schedule types I stuck with this for every mode I did.

B)

Mode 0:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.008017s	N/A	N/A
256	2	0.007828s	N/A	N/A
256	4	0.008213s	N/A	N/A
256	8	0.007923s	N/A	N/A
256	12	0.007889s	N/A	N/A
256	16	0.007776s	N/A	N/A
256	20	0.007883s	N/A	N/A
512	1	0.226314s	N/A	N/A
512	2	0.224801s	N/A	N/A
512	4	0.225640s	N/A	N/A
512	8	0.224283s	N/A	N/A
512	12	0.224646s	N/A	N/A
512	16	0.224780s	N/A	N/A
512	20	0.224983s	N/A	N/A
1024	1	1.888936s	N/A	N/A
1024	2	1.881387s	N/A	N/A
1024	4	1.867399s	N/A	N/A
1024	8	1.866111s	N/A	N/A
1024	12	1.865509s	N/A	N/A
1024	16	1.865922s	N/A	N/A
1024	20	1.896251s	N/A	N/A

Mode 1:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.008186s	0.947204	0.947204
256	2	0.004225s	1.880873	0.940437
256	4	0.002503s	3.201235	0.800309
256	8	0.001762s	4.693149	0.586664
256	12	0.002753s	3.574708	0.297892
256	16	0.002352s	3.909496	0.244343
256	20	0.013743s	0.700632	0.035032
512	1	0.228094s	0.994187	0.994187
512	2	0.118425s	1.946484	0.973242
512	4	0.064443s	3.699513	0.924878
512	8	0.034812s	6.923222	0.865403
512	12	0.030474s	7.722419	0.643535

512	16	0.023879s	9.766855	0.610428
512	20	0.024712s	9.604147	0.480207
1024	1	1.892056s	0.986644	0.986644
1024	2	0.992271s	1.888227	0.944114
1024	4	0.514978s	3.670972	0.917743
1024	8	0.278711s	6.819337	0.852417
1024	12	0.290242s	6.552387	0.546032
1024	16	0.239080s	7.948627	0.496789
1024	20	0.209417s	9.140439	0.457022

Mode 2:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.010024s	0.913181	0.913181
256	2	0.005118s	1.788299	0.894150
256	4	0.002723s	3.438433	0.859608
256	8	0.001577s	6.087979	0.760997
256	12	0.003104s	3.339254	0.278271
256	16	0.002332s	5.279212	0.329951
256	20	0.017542s	0.654233	0.032712
512	1	0.232966s	0.975554	0.975554
512	2	0.118616s	1.970090	0.985045
512	4	0.062806s	3.719929	0.929982
512	8	0.034834s	6.804101	0.850513
512	12	0.028596s	8.104393	0.675366
512	16	0.021813s	10.779856	0.673741
512	20	0.023368s	9.925611	0.496281
1024	1	1.893586s	0.993175	0.993175
1024	2	0.973073s	1.936075	0.968038
1024	4	0.507191s	3.732232	0.933058
1024	8	0.279093s	6.755403	0.844425
1024	12	0.300733s	6.195221	0.516268
1024	16	0.234397s	7.946284	0.496643
1024	20	0.246471s	7.642642	0.382132

Mode 3A:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.008893s	N/A	N/A
256	2	0.006393s	N/A	N/A
256	4	0.004285s	N/A	N/A

256	8	0.004690s	N/A	N/A
256	12	0.006572s	N/A	N/A
256	16	0.006241s	N/A	N/A
256	20	0.029431s	N/A	N/A
512	1	0.251616s	N/A	N/A
512	2	0.138411s	N/A	N/A
512	4	0.077465s	N/A	N/A
512	8	0.043159s	N/A	N/A
512	12	0.043739s	N/A	N/A
512	16	0.035623s	N/A	N/A
512	20	0.032931s	N/A	N/A
1024	1	1.971591s	N/A	N/A
1024	2	1.076049s	N/A	N/A
1024	4	0.580404s	N/A	N/A
1024	8	0.337245s	N/A	N/A
1024	12	0.360327s	N/A	N/A
1024	16	0.308168s	N/A	N/A
1024	20	0.314743s	N/A	N/A

Mode 3B:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.009137s	N/A	N/A
256	2	0.006275s	N/A	N/A
256	4	0.005769s	N/A	N/A
256	8	0.006464s	N/A	N/A
256	12	0.009441s	N/A	N/A
256	16	0.009254s	N/A	N/A
256	20	0.009862s	N/A	N/A
512	1	0.247206s	N/A	N/A
512	2	0.132573s	N/A	N/A
512	4	0.086086s	N/A	N/A
512	8	0.065993s	N/A	N/A
512	12	0.061987s	N/A	N/A
512	16	0.059087s	N/A	N/A
512	20	0.054103s	N/A	N/A
1024	1	2.016763s	N/A	N/A
1024	2	1.063608s	N/A	N/A
1024	4	0.629363s	N/A	N/A
1024	8	0.411729s	N/A	N/A
1024	12	0.449995s	N/A	N/A

1024	16	0.386274s	N/A	N/A
1024	20	0.390224s	N/A	N/A

Mode 4:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.008475s	0.908790	0.908790
256	2	0.004376s	1.811562	0.905781
256	4	0.002289s	3.554195	0.888549
256	8	0.002050s	4.339498	0.542437
256	12	0.002118s	4.406007	0.367167
256	16	0.002434s	3.781686	0.236355
256	20	0.010605s	1.026422	0.051321
512	1	0.235889s	0.969809	0.969809
512	2	0.119144s	1.951462	0.975731
512	4	0.065009s	3.604038	0.901010
512	8	0.034735s	6.859781	0.857473
512	12	0.028470s	8.275620	0.689635
512	16	0.023529s	10.063938	0.628996
512	20	0.023005s	10.143901	0.507195
1024	1	2.028047s	0.972396	0.972396
1024	2	1.049954s	1.908524	0.954262
1024	4	0.542081s	3.717550	0.929387
1024	8	0.299672s	6.653616	0.831702
1024	12	0.262977s	7.781267	0.648439
1024	16	0.206569s	9.817727	0.613608
1024	20	0.178375s	11.437955	0.571898

Mode 5:

N	Threads	Average Time	Speedup	Efficiency
256	1	0.007912s	N/A	N/A
256	2	0.008021s	N/A	N/A
256	4	0.007937s	N/A	N/A
256	8	0.007896s	N/A	N/A
256	12	0.007874s	N/A	N/A
256	16	0.007936s	N/A	N/A
256	20	0.007966s	N/A	N/A
512	1	0.251782s	N/A	N/A
512	2	0.249730s	N/A	N/A
512	4	0.249495s	N/A	N/A

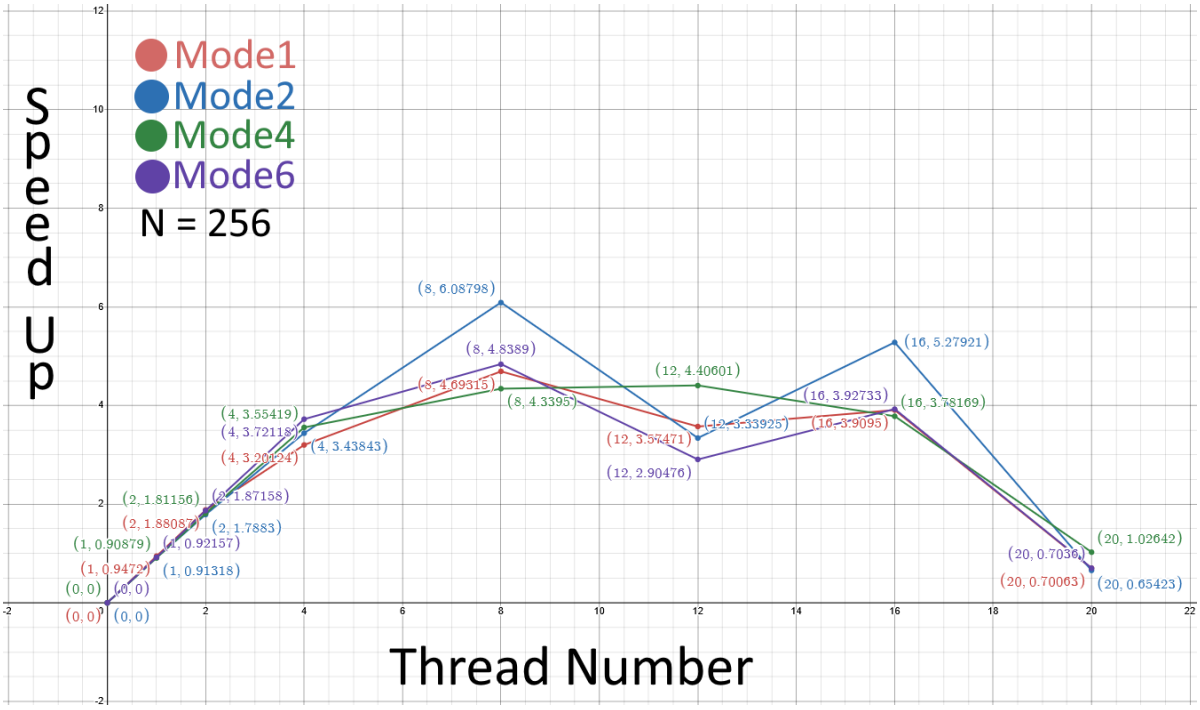
512	8	0.247794	N/A	N/A
512	12	0.247244s	N/A	N/A
512	16	0.246663s	N/A	N/A
512	20	0.245973s	N/A	N/A
1024	1	1.905609s	N/A	N/A
1024	2	1.895951s	N/A	N/A
1024	4	1.930610s	N/A	N/A
1024	8	1.906673s	N/A	N/A
1024	12	1.907455s	N/A	N/A
1024	16	1.892767s	N/A	N/A
1024	20	1.884689s	N/A	N/A

Mode 6:

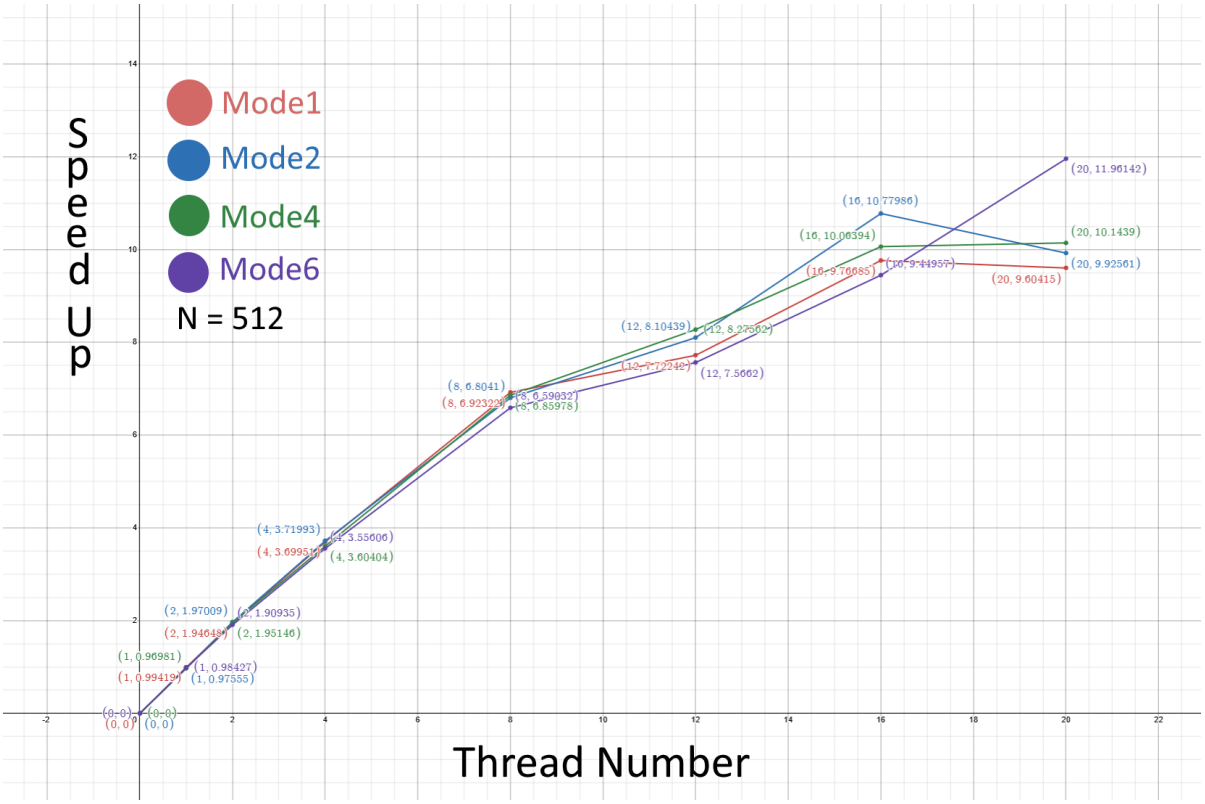
N	Threads	Average Time	Speedup	Efficiency
256	1	0.008265s	0.921567	0.921567
256	2	0.004219s	1.871577	0.935789
256	4	0.002193s	3.721177	0.930294
256	8	0.001714s	4.838896	0.604862
256	12	0.003089s	2.904765	0.242064
256	16	0.002347s	3.927333	0.245458
256	20	0.015360s	0.703603	0.035180
512	1	0.238197s	0.984274	0.984274
512	2	0.121013s	1.909350	0.954675
512	4	0.064574s	3.556059	0.889015
512	8	0.034995s	6.590323	0.823790
512	12	0.030267s	7.566201	0.630517
512	16	0.024516s	9.449572	0.590598
512	20	0.019138s	11.961420	0.598071
1024	1	1.890843s	1.002844	1.002844
1024	2	0.986522s	1.901943	0.950971
1024	4	0.502078s	3.760430	0.940107
1024	8	0.279436s	6.654970	0.831871
1024	12	0.306080s	6.100601	0.508383
1024	16	0.236298s	7.942812	0.496426
1024	20	0.215168s	9.005200	0.450260

C)

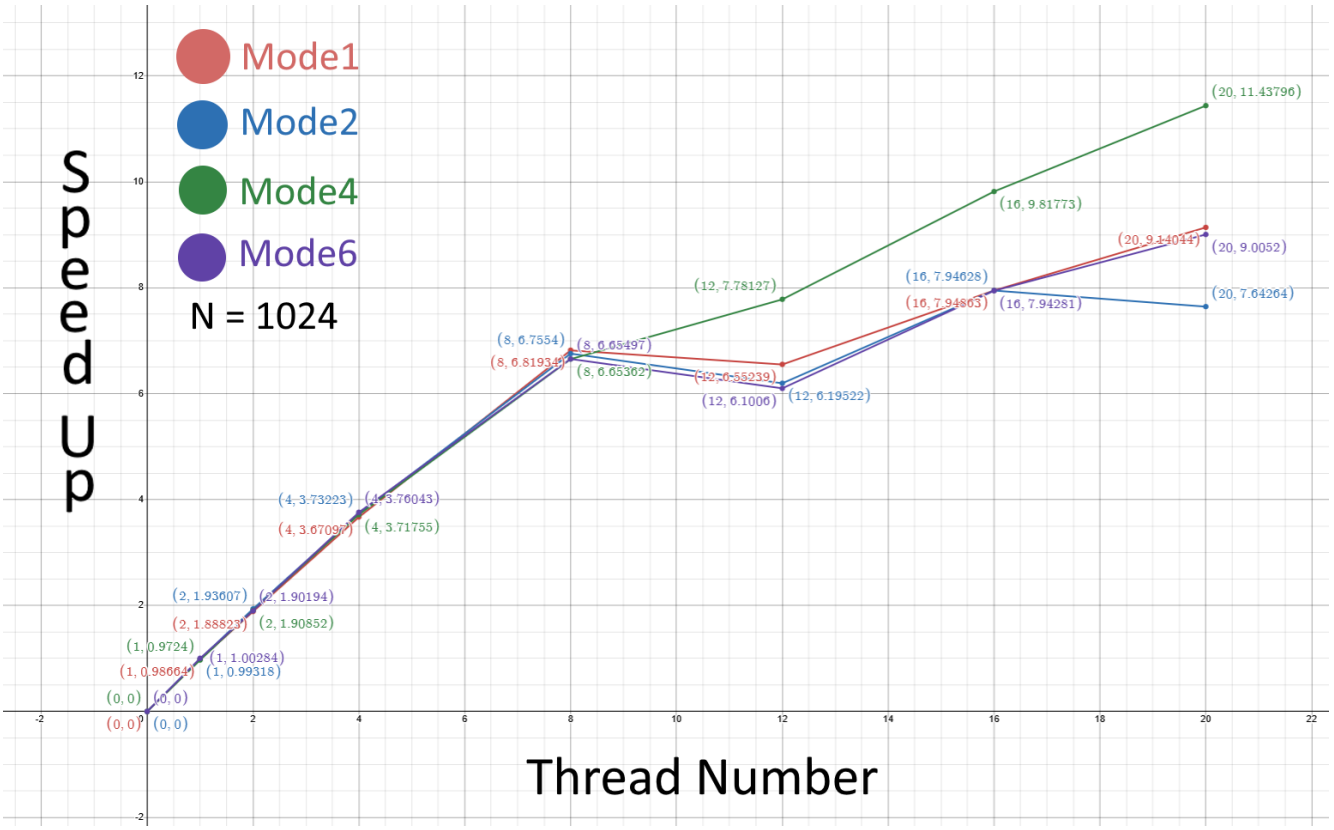
1. N = 256:


Mode1
Mode2
Mode4
Mode6
N = 256
Speed Up
Thread Number
(8, 6.08798)
(8, 4.8389)
(8, 4.69315)
(8, 4.3395)
(16, 5.27921)
(12, 4.40601)
(16, 3.92733)
(16, 3.78169)
(12, 3.57471)
(12, 2.90476)
(4, 3.55419)
(4, 3.72118)
(4, 3.43843)
(4, 3.20124)
(2, 1.81156)
(2, 1.87158)
(2, 1.88087)
(2, 1.7883)
(1, 0.90879)
(1, 0.92157)
(1, 0.9472)
(1, 0.91318)
(0, 0)
(20, 1.02642)
(20, 0.7036)
(20, 0.70063)
(20, 0.65423)

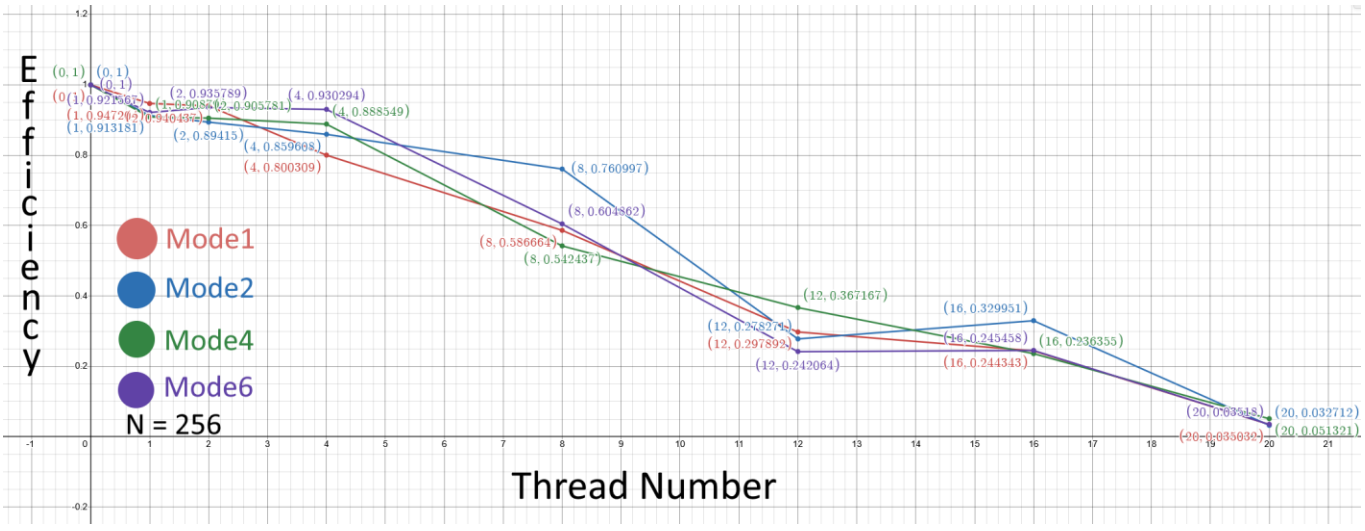
N = 512:


Mode1
Mode2
Mode4
Mode6
N = 512
Speed Up
Thread Number
(20, 11.96142)
(16, 10.77986)
(16, 10.06394)
(20, 10.1439)
(20, 9.92561)
(20, 9.60415)
(12, 8.10439)
(12, 8.27502)
(12, 7.5662)
(8, 6.8041)
(4, 3.71993)
(4, 3.55606)
(4, 3.60404)
(2, 1.97009)
(2, 1.90935)
(2, 1.94648)
(2, 1.95146)
(1, 0.96981)
(1, 0.98427)
(1, 0.97555)
(0, 0)

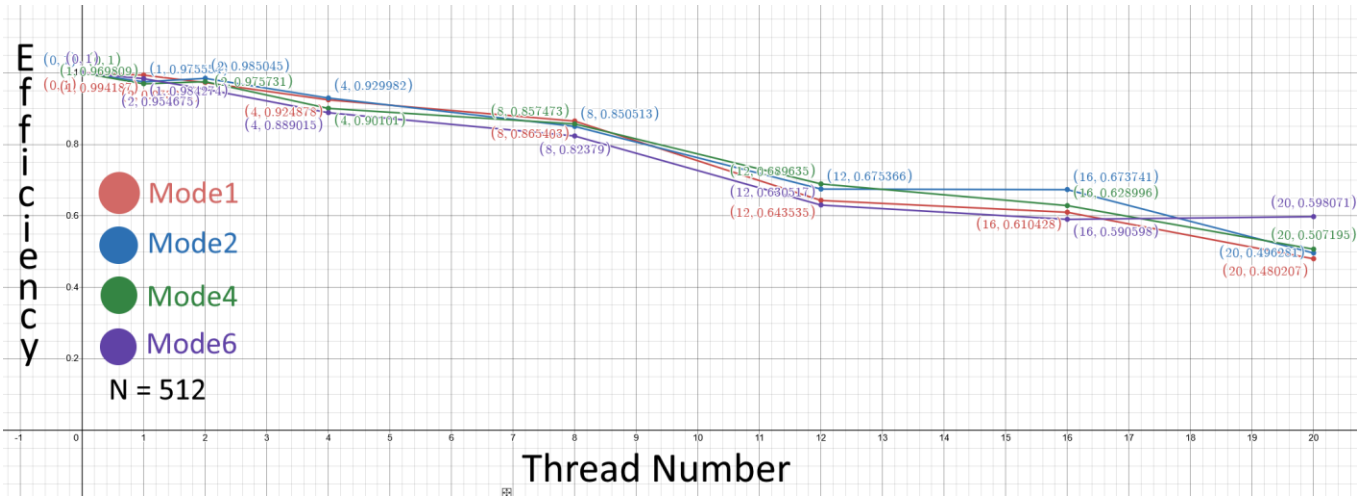
N = 1024:


Mode1
Mode2
Mode4
Mode6
N = 1024
Speed Up
Thread Number

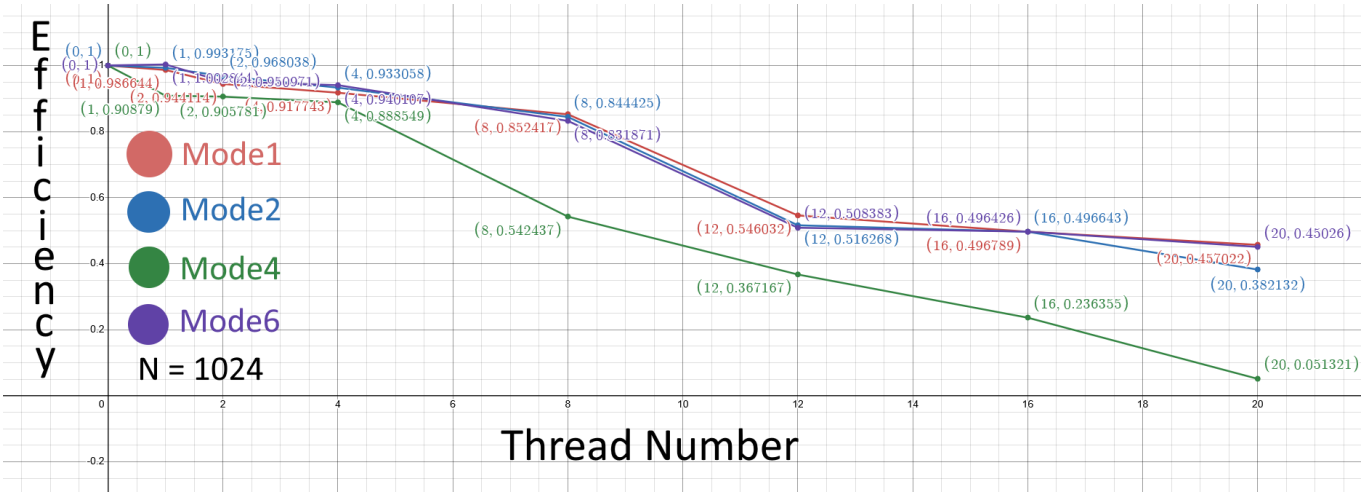
2. N = 256:


Mode1
Mode2
Mode4
Mode6
N = 256
Efficiency
Thread Number

N = 512:



N = 1024:



D)

1. I misunderstood what Mode1 was asking for dynamic vs static scheduling so I ended up doing static and dynamic scheduling for every mode which is why I don't have scheduling in my tables either, sorry.

2. Efficiency drops as the amount of threads increase because of several factors such as high overhead, cpu scheduling, and Amdahl's law. The more threads there are the more serial work that is done, the more time the cpu spends switching between threads, and the higher chance that a thread has poor scheduling.

3. Atomic is faster than Critical when calculating checksum because atomic has much lower overhead than critical does. This is because atomic is designed to do specific operations effieiciently (like +=) when critical is a more general lock so it is slower.

4. At higher N numbers with more threads tasks were way faster than loops at the cost of having very low efficiency compared to loops. At lower N numbers they mostly were the same, but tasks did better when the number of processors was getting too big for the amount of work that needed to be done.

5. Effieiency generally was higher when using SIMD than with just threads, but it didn't make that much of a difference from the data I got. It was also faster than just using threads at higher N levels. It sort of reduced the need for many threads due to being faster than without SIMD, but the difference wasn't big enough to say it is fully worth it.